



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e  
INTERACCIÓN HUMANO COMPUTADORA



## REPORTE DE PRÁCTICA N° 03

**NOMBRE COMPLETO:** Alejandro Santos Jiménez

**N° de Cuenta:** 320325586

**GRUPO DE LABORATORIO:** 13

**GRUPO DE TEORÍA:** 06

**SEMESTRE** 2026-2

**FECHA DE ENTREGA LÍMITE:** 05 de Marzo 2026

**CALIFICACIÓN:** \_\_\_\_\_

## REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

Generar una pirámide rubik (pyraminx) de 9 **pirámides** por cara. Cada cara de la pyraminx que se vea de un color diferente y que se vean las separaciones entre instancias (las líneas oscuras son las que permiten diferenciar cada pirámide pequeña

Para crear esta pirámide mi idea fue crear una cara primero después era copiar todos y cambiar la dirección, pero después le pregunte a la IA si podía hacer eso y mejor me dijo que lo tratara de una función y ya toda una cara la podía ocupar como una función y no lo tenia que hacer manual, entonces fue mas fácil hacer las demás caras y ajustar las coordenadas.

```
void CaraPiramide(glm::mat4 caraPrincipal, glm::vec3 colorFace, GLuint uniformModel, GLuint uniformColor, Mesh* meshPiramide) {  
    glUniform3fv(uniformColor, 1, glm::value_ptr(colorFace));  
    glm::mat4 model;  
    // pico  
    model = glm::translate(caraPrincipal, glm::vec3(0.0f, 1.75f, 0.0f));  
    model = glm::scale(model, glm::vec3(0.9f));  
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
    meshPiramide->RenderMesh();  
  
    //segunda fila  
    model = glm::translate(caraPrincipal, glm::vec3(-0.5f, 0.86f, 0.0f));  
    model = glm::scale(model, glm::vec3(0.9f));  
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
    meshPiramide->RenderMesh();  
  
    model = glm::translate(caraPrincipal, glm::vec3(0.5f, 0.86f, 0.0f));  
    model = glm::scale(model, glm::vec3(0.9f));  
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
    meshPiramide->RenderMesh();  
  
    model = glm::translate(caraPrincipal, glm::vec3(0.0f, 1.2f, 0.0f));  
    model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0.0f, 0.0f, 1.0f));  
    model = glm::scale(model, glm::vec3(0.9f));  
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
    meshPiramide->RenderMesh();  
    //primera fila  
    model = glm::translate(caraPrincipal, glm::vec3(-1.0f, 0.0f, 0.0f));  
    model = glm::scale(model, glm::vec3(0.9f));  
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
    meshPiramide->RenderMesh();  
  
    model = glm::translate(caraPrincipal, glm::vec3(0.0f, 0.0f, 0.0f));  
    model = glm::scale(model, glm::vec3(0.9f));  
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
    meshPiramide->RenderMesh();  
  
    model = glm::translate(caraPrincipal, glm::vec3(1.0f, 0.0f, 0.0f));  
    model = glm::scale(model, glm::vec3(0.9f));  
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
    meshPiramide->RenderMesh();  
  
    model = glm::translate(caraPrincipal, glm::vec3(-0.5f, 0.28f, 0.0f));  
    model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0.0f, 0.0f, 1.0f));  
    model = glm::scale(model, glm::vec3(0.9f));  
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
    meshPiramide->RenderMesh();  
}
```

Creación de cada cara:

```

// 1. Matriz global que controla todo el objeto para moverlo
glm::mat4 cuboRubik = glm::mat4(1.0f);
cuboRubik = glm::translate(cuboRubik, glm::vec3(0.0f, -0.2f, -5.0f));
cuboRubik = glm::rotate(cuboRubik, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
cuboRubik = glm::rotate(cuboRubik, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
cuboRubik = glm::rotate(cuboRubik, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(cuboRubik));

float inclinacionTetraedro = -19.45f;

// --- CARA 1 (FRENTE - VERDE) ---
glm::mat4 cara1 = glm::rotate(cuboRubik, glm::radians(0.0f), glm::vec3(0.0f, 1.0f, 0.0f));
cara1 = glm::rotate(cara1, glm::radians(-19.45f), glm::vec3(1.0f, 0.0f, 0.0f));
cara1 = glm::translate(cara1, glm::vec3(0.0f, -2.5f, 0.0f));
CaraPiramide(cara1, glm::vec3(0.0f, 0.9f, 0.1f), uniformModel, uniformColor, meshList[1]);

// --- CARA 2 (DERECHA - ROJA) ---
glm::mat4 cara2 = glm::rotate(cuboRubik, glm::radians(120.0f), glm::vec3(0.0f, 1.0f, 0.0f));
cara2 = glm::rotate(cara2, glm::radians(-20.85f), glm::vec3(1.0f, 0.0f, 0.0f));
cara2 = glm::translate(cara2, glm::vec3(0.0f, -2.5f, 0.0f));
CaraPiramide(cara2, glm::vec3(0.9f, 0.0f, 0.0f), uniformModel, uniformColor, meshList[1]);

// --- CARA 3 (IZQUIERDA - AZUL) ---
glm::mat4 cara3 = glm::rotate(cuboRubik, glm::radians(240.0f), glm::vec3(0.0f, 1.0f, 0.0f));
cara3 = glm::rotate(cara3, glm::radians(-20.85f), glm::vec3(1.0f, 0.0f, 0.0f));
cara3 = glm::translate(cara3, glm::vec3(0.0f, -2.5f, 0.0f));
CaraPiramide(cara3, glm::vec3(0.0f, 0.0f, 0.9f), uniformModel, uniformColor, meshList[1]);

// --- CARA 4 (BASE AMARILLA) ---
glm::mat4 cara4 = glm::rotate(cuboRubik, glm::radians(180.0f), glm::vec3(0.0f, 1.0f, 0.0f));
cara4 = glm::translate(cara4, glm::vec3(0.0f, -2.7f, 0.0f));
cara4 = glm::rotate(cara4, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
cara4 = glm::translate(cara4, glm::vec3(0.0f, -0.57735f, 0.0f));
CaraPiramide(cara4, glm::vec3(1.0f, 1.0f, 0.0f), uniformModel, uniformColor, meshList[1]);

// parte negra donde se incrusta lo demas

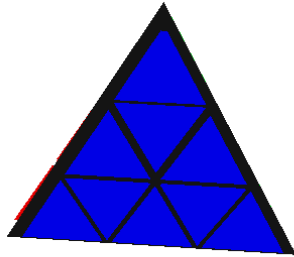
glm::mat4 nucleo = cuboRubik;
nucleo = glm::rotate(nucleo, glm::radians(-19.45f), glm::vec3(1.0f, 0.0f, 0.0f));
float escalaNucleo = 3.25f;
float bajarPunta = 0.57735027f * escalaNucleo;
nucleo = glm::translate(nucleo, glm::vec3(0.0f, -bajarPunta, -0.02f));
nucleo = glm::scale(nucleo, glm::vec3(escalaNucleo));
glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(0.08f, 0.08f, 0.08f)));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(nucleo));
meshList[1]->RenderMesh();

/*

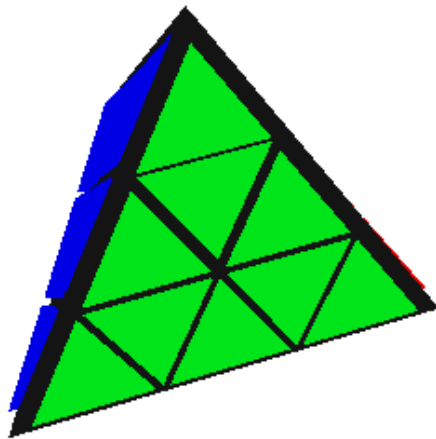
```

Primera

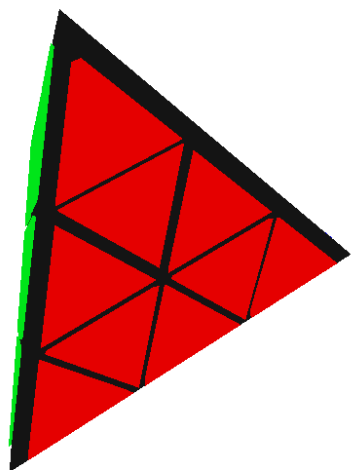
cara:



Segunda cara:

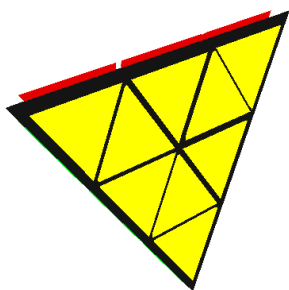


Tercera cara:



Cuarta

cara:



### 3.- Conclusión:

- a. Los ejercicios del reporte: Complejidad, Explicación.

Esta practica tenía dos complejidades donde se podía ocupar menos triángulos para crear la pirámide o hacerlo mas fácil con mas triángulos yo opte con la segunda.

- b. Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias para mejorar desarrollo de la práctica

La practica me gusto estaba laboriosa en general bien.

### 1. Bibliografía en formato APA